Consensus over Orbits: Candidate Selection as a First-Class Variable for Test-Time Training on ARC-AGI-2

zjukop1 (ARC Prize 2026, Paper Track)

November 2026

Linked code submission: ARC-AGI-2 track, ref 55 904 132 (public LB 30.97; local eval 32.3/117), selector kgmon; control run ref 55 679 855 (public LB 31.39), unmodified public baseline. **Code and artifacts:** https://github.com/zjukop3/arc-prize-2026-ttt-consensus (our contributions CC0; derived pipeline under the original notebook's license).

Abstract

ARC-AGI tasks are deliberately under-determined: a handful of training pairs rarely pins down a unique transformation, so a solver must resolve ambiguity at test time. Test-time training (TTT) addresses the model side of this problem; the *selection* side — given dozens of candidate grids produced under symmetry augmentations, which two do you submit? — is usually an undocumented heuristic. We isolate candidate selection as an independent variable on a fixed public TTT backbone (Qwen3-4B + per-task LoRA, 4×L4, 12 h window), benchmarking five selection strategies on a single shared candidate store from one full 117-puzzle local evaluation. Findings: (i) a vote-count consensus rule (kgmon) is the strongest selector (32.3/117) and its solved set strictly contains the probability-product baseline's (31.3/117); (ii) contrary to our stated prediction, interleaving the two rankings adds nothing (identical solved set), and explicit second-attempt diversification *hurts* slightly (31.2); (iii) the oracle union of all five selectors covers 45 solved puzzle-keys versus 44 for the best single rule — selection on this candidate store is nearly saturated, so further gains must come from candidate *generation*, not ranking. The selector spread (30.2–32.3, ±1.8 pp) is nonetheless larger than many published training-side tweaks, at zero compute cost. We publish the full pipeline, the candidate store schema, and the budget envelope (5.4 h for 120 tasks on 4×L4) that makes the ablation reproducible.

1. Introduction

ARC-AGI measures fluid intelligence: each task presents a few input–output grid pairs, and the solver must infer a transformation and apply it to test grids. ARC-AGI-2 (2025) raised the difficulty of the static benchmark; ARC-AGI-3 (2026) moved to interactive environments with step budgets.

Public progress on ARC-AGI-2 has converged on a recipe: serialize grids as text, fine-tune a mid-size LLM on ARC-like data, adapt per task at test time, and decode multiple candidate solutions under symmetry augmentations. Community discussion focuses almost entirely on the training side — augmentation counts, LoRA settings, decoding thresholds. The last step of

the pipeline, **choosing attempt_1 and attempt_2 from the candidate pool**, is treated as glue code.

This paper treats it as a variable. On a fixed backbone we vary only the selector and measure pass rates on the full evaluation set. Contributions:

1. **A controlled ablation of five selection strategies** on one shared candidate store (117 puzzles, 2 attempts) — to our knowledge the first public isolation of candidate selection in ARC-AGI.
2. **A negative result with a mechanism.** We predicted that confidence-based and vote-based rankings would carry orthogonal signal and that interleaving them would help. The data falsifies this: interleaving reproduces the confidence ranking's solve set exactly, while pure vote consensus strictly dominates it. §4 offers the revised account.
3. **A saturation bound.** The oracle union of all five selectors adds ≈1 fractional task over the best single rule — selection headroom on this candidate store is nearly exhausted; generation diversity is the binding constraint.
4. **A reproducible budget envelope** for TTT on Kaggle's 4×L4 / 12 h constraint, including the GPU-side 12-token DFS change that removes the CPU bottleneck.

Our leaderboard scores are mid-table by design: the linked submission differs from the public baseline only in selection and systems code. The contribution is methodological, and the assets are open sourced.

2. Prior work

ARC-AGI. Chollet (2019) introduced the benchmark as a measure of abstraction and reasoning efficiency. The ARC Prize 2026 edition pairs a static track (ARC-AGI-2) with an interactive track (ARC-AGI-3).

Test-time training for ARC. Akyürek et al. (2024) showed per-task fine-tuning at inference substantially improves LLM performance on ARC-AGI-1. TTT is not our contribution. Our backbone is the public LB-33.89 baseline notebook by Kaggle user mikelou1 ("ARC AGI2 LB33.89 Minimal Perfpatch", itself derived from Koushik Rudra's AIMO-series notebook), which contributes the pipeline skeleton: the grid-serialized Qwen3-4B SFT model (sorokin/qwen3_4b_grids15_sft139), LoRA configuration, turbo-DFS decoding, and teacher-forced candidate scoring described in §3.1.

Program synthesis. DreamCoder (Ellis et al., 2021) and successors solve ARC-like tasks by searching a discrete program space. Our approach is neural; search is restricted to a 12-symbol output alphabet.

Self-consistency. Wang et al. (2022) sample chains and majority-vote. Our setting differs: votes are collected over *symmetry-group augmentations* of a fixed task rather than i.i.d. samples, and the ARC metric allows two scored attempts, which invites explicit diversification.

What is new. No public ARC-AGI work we could find isolates candidate selection as an independent variable with controlled comparisons on a fixed backbone. The selectors we

benchmark are deliberately simple; the contribution is the measurement, the falsified hypothesis, and the saturation bound.

3. Approach

3.1 Backbone (inherited; described for completeness)

Model. Qwen3-4B SFT'd on grid-serialized ARC-like data (sorokin/qwen3_4b_grids15_sft139), bf16, max sequence length 8192.

Per-task LoRA TTT. For each task the model adapts on that task's training pairs only: LoRA rank 256 over all attention/MLP projections plus embeddings and head, α=32, rsLoRA, lr 5e-5, cosine, 1 epoch, batch size 1, AdamW. Training data = training pairs × 16 augmentations (dihedral × color-permutation group actions, shuffled keys). Adapter weights are snapshotted before and restored after each task.

Decoding and scoring. Candidates are generated by DFS restricted to the 12 ARC color/format tokens ("turbo-DFS") with NLL-threshold pruning (max_score = −ln 0.2); a teacher-forced pass (calc_scores) then scores each candidate under further augmentations. Each candidate record stores: grid, beam score, and augmented-NLL vector (score_aug). All selectors below are deterministic functions of these records — selection never re-runs the model.

3.2 Candidate selection as the independent variable

Given a task's candidate multiset (each candidate carrying vote count = number of decoding paths that produced it, beam score, augmented NLLs), we benchmark five selectors:

- **probmul_3** — probability product: Σ(3 − beam) + mean_g Σ(3 − s_aug). Confidence-heavy.
- **kgmon** — vote count, tie-broken by mean augmented NLL. Consensus-heavy.
- **ensemble** — round-robin interleave of the probmul and kgmon rankings, deduplicated.
- **portfolio** — hand-weighted blend: 2.0·votes − 0.35·aug_mean − 0.15·beam − 0.10·aug_best.
- **ensemble_plus** — attempt_1 from the ensemble ranking; attempt_2 = first *distinct* candidate under portfolio, then fallbacks.

Tasks with multiple test grids are split per grid and re-aggregated to match the official metric (a puzzle counts only if all its grids are solved; ≤2 attempts per grid; 117 puzzles with multi-grid puzzles contributing fractionally).

3.3 Budget accounting (engineering note)

The pipeline runs on 4×L4 in Kaggle's 12-hour window with a 10-minute write buffer. Two changes made the parallelism real:

1. **GPU-side logit normalization for turbo-DFS.** The baseline normalized full-vocabulary logits on CPU; we normalize on GPU and ship only the 12 retained token scores to CPU, removing the dominant stall so four workers (one per L4) saturate.
2. **GPU teacher-forced scoring** with KV cache disabled on unconsumed branches (calc_scores), avoiding quadratic memory growth over augmented queries.

Measured envelope: the full 120-task evaluation (117 puzzles after multi-reply splitting) completes in **5.4 h wall-clock (≈2.7 min/puzzle), ≈45% of the window** — leaving headroom for roughly a 2× increase in augmentation or model scale before the budget binds. A control run of the unmodified baseline (ref 55 679 855) scores 31.39 public vs our build's 30.97 (v39 changes selection and systems code only); the gap is within the run-to-run variance we observe between local (32.3/117) and public evaluation.

3.4 Universality: the same principle in ARC-AGI-3

In the interactive track, "adapt then reach consensus under a budget" becomes a *budget-aware search agent* (Pathfinder v2): a per-game time-budget allocator, background-threaded solving so action selection never blocks, and sum-tree prioritized replay of experience. Public LB 0.25 — a principle-level transfer, not a competitive score: the static track's consensus over augmented views corresponds, in the interactive setting, to spending search where the remaining budget is scarcest. We include it as evidence about the principle's scope, not the score's.

4. Why consensus works — and why ensembling didn't

ARC tasks are under-determined by design: training pairs admit multiple consistent transformations, so single-view decoding is ill-posed. Symmetry augmentation converts this ambiguity into something measurable.

Let G be the task's symmetry group (dihedral × color permutations). Scoring candidate c under augmented views g·task is a Monte-Carlo estimate of the orbit-marginal E_g[log p(c | g·task)], and the vote count estimates the size of c's basin of attraction — the fraction of independently augmented decoding paths that reach c at all. Confidence (probmul) and popularity (votes) measure different things, so we predicted interleaving them would recover tasks either parent missed.

The data says no. On the shared candidate store, ensemble's solved set is *identical* to probmul's, while kgmon's strictly contains it (adding task 6e453dd6_0). The two rankings are not orthogonal where it matters: for candidates near the decision boundary (top-2), the confidence order is essentially a monotone transform of the vote order. The refined account: once decoding is run over enough augmentations, basin size dominates — a candidate reached from many augmented starting points is almost always the orbit-stable mode, and confidence adds no independent evidence. Consistent with this, portfolio (a fixed linear blend) underperforms both pure rules, and ensemble_plus's forced diversification of attempt_2 spends the second attempt on worse candidates twice (losing 3a25b0d8_0, a395ee82_0 relative to probmul), costing 0.1 task net.

The useful prediction that survives: selection gains should concentrate where the candidate set is diverse. Here they don't materialize because the store is *not* diverse near the top — which is exactly the saturation diagnosis of §5.

5. Results

Main results (official leaderboard).

Track	Submission	Public LB	Local eval	Note
ARC-AGI-2	ref 55 679 855	31.39	—	unmodified public baseline (control)
ARC-AGI-2	ref 55 904 132 (v39, kgmon)	30.97	32.3 / 117	linked submission
ARC-AGI-2	ref 55 671 569 (enumeration baseline)	0.00	—	no-TTT lower bound
ARC-AGI-3	ref 55 757 351 (Pathfinder v2)	0.25	—	principle transfer only

Selection ablation. One TTT run produced one candidate store (full 120-task evaluation set, 117 puzzles after multi-grid splitting, 2 attempts); selectors differ only in post-processing of identical artifacts.

Selector	Solved (of 117)	Puzzle-keys solved	Δ vs probmul_3
kgmon	**32.3**	44	+1.0
probmul_3	31.3	43	—
ensemble	31.3	43 (identical set to probmul_3)	0.0
ensemble_plus	31.2	43	−0.1
portfolio	30.2	42	−1.1
oracle union of all five	≈33.3*	45	≈ +1.0 vs best single

* fractional estimate from puzzle-key coverage; the official metric re-aggregates multi-grid puzzles.

Candidate statistics. 394 of 2339 candidate records (16.8%) are exactly correct; correct candidates carry beam scores from 0.271 (avg) to 1.553 (max), confirming that confidence alone cannot identify them — the best selector's job is ranking, not filtering. The selector spread of 2.1 tasks (30.2–32.3) is the size of a respectable training-side tweak, obtained by pure post-processing; but the oracle-union bound shows selection on this store is within ≈1 task of saturated.

Failure modes. 83% of generated candidates are wrong, and per-puzzle coverage (not ranking) is the constraint: for most unsolved puzzles the correct grid is simply absent from the candidate store — no selector can recover it. We did not cluster unsolved tasks by type; we report the aggregate honestly and note that wrong-size outputs ("bad_xy_size") and wrong-content outputs are both terminal under exact-match scoring. The public↔local gap (30.97 vs 32.3/117-scaled) is consistent with ordinary sample noise across different task sets; we do not read more into it.

We report only public-leaderboard and held-out local evaluation numbers, never training-set performance.

6. Conclusion

On a fixed public TTT backbone, the choice of candidate selector moves ARC-AGI-2 accuracy by ±1.8 pp at zero training cost — and then stops: the oracle union of five reasonable selectors is within ≈1 task of the best single rule. The practical recommendation is boring but actionable: **use vote-count consensus (kgmon), and stop tuning selection — spend the effort on candidate generation diversity**, where the headroom provably lives. For the 85% goal, the reusable assets are the method (isolate selection on a shared candidate store), the negative result (confidence and vote rankings are not orthogonal near the decision boundary), and the open pipeline with its measured 4×L4 budget envelope.

Limitations. Everything inherits the ceiling of a 4B SFT backbone. The ablation is a single run on one store; selectors are deterministic given the store, but the store reflects one TTT run's sampling. The ARC-AGI-3 evidence is principle-level (0.25). The saturation bound is measured on one backbone; stronger generators may reopen selection headroom — which our harness measures cheaply, by design.

References

1. Chollet, F. (2019). On the Measure of Intelligence. arXiv:1911.01547.
2. Akyürek, E., et al. (2024). The Surprising Effectiveness of Test-Time Training for Abstract Reasoning. arXiv:2411.07279.
3. Wang, X., et al. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171.
4. Ellis, K., et al. (2021). DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning. Phil. Trans. R. Soc. A 381.
5. mikelou1 (2026). *ARC AGI2 LB33.89 Minimal Perfpatch* (public baseline notebook, derived from Koushik Rudra's "Failed in AIMO"). Kaggle. https://www.kaggle.com/code/mikelou1/arc-agi2-lb33-89-minimal-perfpatch
6. sorokin (2026). *qwen3_4b_grids15_sft139* (model card). Kaggle Models.
7. ARC Prize Foundation (2026). ARC-AGI-2 / ARC-AGI-3 documentation. https://arcprize.org